

A Leakage-Aware Machine Learning Approach to Content Refresh Prioritization

Detecting and Correcting Data Leakage, Group-Based Validation, and Out-of-Fold Scoring for Editorial Content Maintenance

Capstone Project Report

Abstract

Search-engine-optimized (SEO) content decays in relevance as search intent, competitive rankings, and topical trends shift over time. Content teams managing large catalogs of articles need a systematic way to identify which pieces of content most urgently require revision. This report documents the design, implementation, and evaluation of a Content Refresh Prioritization Engine: a supervised machine learning system that scores content by its estimated need for refresh, using a compact set of “content DNA” features available at authoring time. We describe a rule-based labeling scheme derived from trend, search-volume, and staleness signals; a client-grouped train/test protocol designed to prevent cross-client leakage; and a comparison between Logistic Regression and Random Forest classifiers. A central contribution of this work is methodological rather than purely predictive: during development we identified and corrected two forms of data leakage — (1) age-derived features that were mechanically entangled with the label definition, and (2) an evaluation procedure that scored the full dataset with a model that had already been trained on part of it. We present the corrected pipeline, its measured performance (ROC-AUC of 0.891 for Random Forest and 0.879 for Logistic Regression on a held-out set of 8 unseen clients), feature importance analysis, and an out-of-fold scoring procedure suitable for generating an unbiased content priority leaderboard. We conclude with a discussion of limitations, including class imbalance, the small number of distinct clients available for group-based validation, and directions for future work.

1. Introduction

1.1 Motivation

Organizations that publish and maintain large volumes of web content face a recurring operational question: given finite editorial capacity, which articles should be updated first? Content that once ranked well for a given search query can lose relevance as search interest declines, as competitors publish more current material, or as the underlying topic becomes stale. Manually auditing thousands of articles for refresh candidacy does not scale, motivating an automated, data-driven prioritization approach.

This project frames refresh prioritization as a binary classification problem: predict whether a given piece of content currently “needs refresh,” and convert the model’s predicted probability into a continuous priority score that can be used to rank an entire content catalog.

1.2 Problem Statement

Formally, given a piece of content described by a set of static, authoring-time attributes (e.g., word count, competition level, search intent, content type), we aim to predict a binary outcome, `needs_refresh`, defined by a composite rule over trend percentage, search volume, and staleness (days since last update).

The predictive model must generalize not only to unseen articles but to entirely unseen clients, since new clients onboarded to the platform will have no historical labeled data of their own.

1.3 Contributions

- A leakage-safe feature set (“content DNA”) that excludes any attribute mechanically derived from the same signals used to build the label.
- A documented case study of two leakage failure modes discovered during iterative development, and the corrections applied to each.
- A client-grouped train/test and cross-validation protocol that evaluates generalization to unseen clients rather than unseen rows.
- An out-of-fold scoring procedure that produces an unbiased priority leaderboard across the full dataset, avoiding self-evaluation bias.
- An empirical comparison of Logistic Regression and Random Forest classifiers, with feature importance analysis and threshold-selection guidance for production use.

2. Related Work

Content decay and “content freshness” have been widely discussed in the SEO practitioner literature, which generally recommends periodic manual audits triggered by ranking or traffic drops. This project differs by casting the decision as a supervised learning problem, enabling proactive, ranked prioritization across an entire catalog rather than reactive, single-article review.

From a methodological standpoint, this work draws on two well-established practices in applied machine learning. First, group-based cross-validation (e.g., GroupKFold, GroupShuffleSplit) is standard practice whenever multiple observations share a common entity — here, client — to avoid optimistic bias from entity-level leakage across the train/test boundary. Second, out-of-fold prediction (as used in stacked generalization and honest model evaluation) ensures that no observation is scored by a model that was trained on it, which is essential whenever model scores themselves are the deliverable, as in this project’s priority leaderboard.

3. Data and Label Construction

3.1 Dataset Description

The dataset consists of anonymized content records spanning multiple clients, each row representing a single piece of published content. After removing rows with missing values in the fields required to construct the label (described in Section 3.2), the working dataset contained 25,287 labeled rows spanning 31 distinct clients (23 clients used for training and 8 held out for testing, as detailed in Section 4.2).

Each record includes numeric attributes (e.g., competition score, cost-per-click, word count, character count), categorical attributes (e.g., competition level tier, content type, primary search intent, the AI provider and model used to assist authoring), and identifying fields (content_id, client_id) that are excluded from the feature set but used for grouping and reporting.

3.2 Rule-Based Label Definition

The target variable, `needs_refresh`, is constructed from three underlying signals rather than collected as a direct annotation. A content item is labeled positive (1) if all of the following hold simultaneously:

- `trend_pct` $\leq -20\%$ (search interest for the topic has declined by at least 20% relative to its baseline);
- `search_volume` > 0 (the topic still has non-zero search demand, ruling out topics that are simply obsolete);
- `days_since_last_update` ≥ 30 (the content has not been touched in at least 30 days, so a very recent update is not flagged again immediately).

This composite rule reflects a practical editorial heuristic: refresh effort should be directed at content that is both declining in relevance and still worth the investment, and that has not already been recently addressed. Under this definition, the overall positive rate in the dataset is 11.5%, indicating a moderately imbalanced classification problem that informed the modeling choices in Section 4.4.

3.3 Handling Missing Values in Label Construction

An early version of the labeling code called a missing-value filter (dropna) on the `needs_refresh` column only after that column had already been computed from boolean comparisons. Because a boolean comparison against a missing value evaluates to False rather than propagating a missing value, that filter was a silent no-op: rows with missing `trend_pct`, `search_volume`, or `days_since_last_update` were never actually excluded — they were instead mislabeled as `needs_refresh = 0`. This inflates the apparent negative class and introduces label noise correlated with data completeness, which can itself be informative (and therefore a leakage risk) if missingness is not missing at random.

The corrected pipeline explicitly checks the three label-input columns for missing values and removes affected rows before the label is computed, ensuring that every labeled row reflects a genuine evaluation of the refresh rule rather than a default value produced by an unintentionally permissive comparison.

4. Methodology

4.1 Feature Engineering and Leakage Mitigation

The feature set is restricted to what we term “content DNA”: attributes describing the intrinsic characteristics of a piece of content that are available at, or shortly after, authoring time — competition, cost-per-click, word count, character count, competition-level tier, content type, primary search intent, word-count tier, character-count tier, content provider, and the generative model used during authoring.

An earlier iteration of the feature set also included `content_age_days`, `age_tier`, and `age_tier_order`. Because the label definition directly incorporates `days_since_last_update`, and content age is mechanically related to that same staleness signal, including age-derived features let the model substantially reconstruct one clause of its own label rather than learn a generalizable relationship. This was detected empirically: in the leaking configuration, `content_age_days` and several age-tier indicator features dominated the Random Forest feature-importance ranking, a pattern characteristic of label leakage rather than genuine predictive signal. All age-derived features were subsequently removed from the model input; Section 5.2 shows the corrected importance ranking, in which competition, word count, and character count become the leading predictors.

4.2 Group-Based Train/Test Splitting

Rather than a random row-level split, the dataset is partitioned using `GroupShuffleSplit` keyed on `client_id`, so that all content belonging to a given client falls entirely within either the training set or the test set, never both. This design choice reflects the deployment reality that the model must generalize to clients it has never observed during training, and prevents an optimistic bias that would result from a client's idiosyncratic patterns appearing in both partitions. The resulting split allocated 19,707 rows across 23 clients to training and 5,580 rows across 8 clients to testing (`test_size = 0.25`, `random_state = 42`).

4.3 Preprocessing Pipeline

Numeric features are median-imputed and standardized; categorical features are constant-imputed with an explicit “unknown” category and one-hot encoded with unknown categories ignored at inference time. Both transformations are composed with a scikit-learn `ColumnTransformer` inside a single `Pipeline` object per model, ensuring that preprocessing statistics (e.g., medians, encoded category vocabularies) are fit exclusively on training data and applied without refitting to the test set.

4.4 Model Selection

Two classifiers were trained and compared under identical preprocessing: Logistic Regression (`class_weight = “balanced”`, `max_iter = 1000`) as an interpretable linear baseline, and Random Forest (300 trees, `max_depth = 8`, `class_weight = “balanced”`) as a non-linear ensemble capable of capturing feature interactions. Class weighting was applied in both models to counteract the 11.5% positive rate identified in Section 3.2, penalizing misclassification of the minority (`needs_refresh = 1`) class more heavily during training.

4.5 Evaluation Metrics

Because the positive class is a minority and the operational cost of a missed refresh differs from the cost of a false alarm, we report precision, recall, and F1-score per class in addition to ROC-AUC, which is threshold-independent and therefore useful for comparing model discriminative power before a specific operating threshold is chosen for deployment (see Section 6.2).

5. Experimental Results

5.1 Classification Performance

Table 1 summarizes held-out test performance (8 unseen clients, 5,580 rows) for both classifiers under the corrected, leakage-free feature set.

Metric	Logistic Regression	Random Forest
Precision (class 0)	0.955	0.960
Recall (class 0)	0.977	0.959
Precision (class 1)	0.502	0.419
Recall (class 1)	0.336	0.427

Metric	Logistic Regression	Random Forest
F1-score (class 1)	0.403	0.423
Accuracy	0.935	0.924
ROC-AUC	0.879	0.891

Table 1. Held-out test-set performance by model (client-grouped split, 8 test clients).

Random Forest achieves the higher ROC-AUC (0.891 vs. 0.879) and the better F1-score on the positive class, while Logistic Regression achieves higher precision on the positive class at the default 0.5 threshold. In both cases, precision on the minority class (0.42–0.50) indicates that, at a default threshold, a substantial fraction of flagged content would not in fact need refresh under the labeling rule — a consideration addressed via threshold tuning in Section 6.2 rather than by the raw classifier output alone.

5.2 Feature Importance Analysis

Table 2 lists the top features by Gini importance from the Random Forest model, computed on the corrected, leakage-free feature set.

Rank	Feature	Importance
1	competition	0.129
2	word_count	0.117
3	char_count	0.115
4	main_intent_informational	0.088
5	word_count_tier_1000–2000	0.076
6	char_count_tier_15000–25000	0.068
7	word_count_tier_2000–3500	0.058
8	model_used_gpt-4o-mini	0.045
9	model_used_gemini-3-flash-preview	0.036
10	main_intent_transactional	0.036

Table 2. Top 10 Random Forest feature importances after removing age-derived, leakage-prone features.

Competitive intensity and content length dominate the corrected ranking, followed by search intent and the specific AI model used during content generation. This is substantively more plausible as a genuine predictive signal than the pre-correction ranking, in which age-derived features accounted for the majority of importance mass.

5.3 Discussion of Leakage Detection and Correction

This project's most instructive finding is procedural rather than statistical: the feature-importance ranking itself served as a diagnostic for leakage. When `content_age_days` and its tiered derivatives dominated the importance ranking despite being drawn from a small, mechanically-defined set of “content DNA”

features, this was a signature of the model exploiting its own label definition rather than learning transferable structure. We recommend that any classifier trained on a rule-derived label routinely audit feature importance for exactly this pattern before accepting reported performance at face value.

6. Prioritization Scoring for Production Use

6.1 Out-of-Fold Scoring to Avoid Self-Evaluation Bias

The end deliverable of this system is not a single train/test accuracy figure but a continuous `refresh_priority_score` for every piece of content in the catalog, used to rank and triage editorial work. An initial implementation computed this score by calling `predict_proba` on the full feature matrix using the model trained on the training split alone — which meant that rows belonging to training clients received predictions from a model that had already observed their labels, inflating their scores relative to rows from held-out clients and producing a leaderboard that did not reflect genuine out-of-sample priority.

The corrected procedure uses `GroupKFold` cross-validation together with `cross_val_predict(method="predict_proba")` across the same client grouping used for evaluation, so that every row in the dataset receives a probability from a model instance that never saw that row's client during training. This yields an unbiased, catalog-wide priority score suitable for ranking content across all clients, including those used in earlier training folds.

6.2 Threshold Selection Considerations

Because editorial capacity is finite, the operating threshold applied to `refresh_priority_score` should be chosen based on the volume of recommendations a content team can realistically act on, rather than accepting the default 0.5 decision boundary. A precision-recall curve computed on the held-out test set allows selection of a threshold that meets a target precision (e.g., at least 50% of flagged items genuinely meeting the refresh rule) or a target recommendation volume (e.g., the top N items per week), and this threshold should be re-validated whenever the model is retrained on new data.

7. Limitations

- Client population size: with only 31 distinct clients, group-based cross-validation folds are coarse, and performance estimates may be sensitive to which specific clients fall into a given fold.
- Label circularity: the target is itself rule-derived rather than independently annotated, so the model is learning to approximate a heuristic rather than a ground-truth editorial judgment; if the heuristic is later revised, the model must be retrained and re-validated rather than assumed to remain valid.
- Class imbalance: even with class weighting, precision on the positive class remains moderate (0.42–0.50 at the default threshold), meaning a non-trivial share of flagged content will not need refresh under the labeling rule.
- Static feature snapshot: features reflect a snapshot of content and market attributes and do not capture time-series dynamics (e.g., the trajectory of `trend_pct` over time) that could improve early detection of decline.

8. Future Work

- Incorporate time-series features (e.g., rolling trend slope, rate of search-volume change) rather than single-point-in-time snapshots.
- Expand the client pool to stabilize group-based cross-validation estimates and enable stratification by client size or vertical.
- Calibrate predicted probabilities (e.g., via Platt scaling or isotonic regression) so that `refresh_priority_score` can be interpreted as an estimated likelihood rather than only a ranking signal.
- Separate the training/evaluation pipeline from a lightweight production inference script that loads a persisted model artifact, so that scheduled scoring runs do not repeat cross-validation or threshold search on every invocation.
- Conduct a prospective evaluation: track whether content flagged as high-priority and subsequently refreshed shows measurable recovery in `trend_pct` or search performance.

9. Conclusion

This report documented a supervised learning pipeline for prioritizing content refresh decisions, built on a rule-derived label and a compact, authoring-time feature set. Beyond model comparison, the project's central contribution is a documented process for detecting and correcting two forms of data leakage — feature leakage through age-derived attributes mechanically tied to the label, and evaluation leakage through scoring the full dataset with a model that had already trained on part of it. With both issues corrected, the Random Forest classifier achieves a held-out ROC-AUC of 0.891 under a client-grouped validation protocol designed to reflect real-world generalization to unseen clients, and an out-of-fold scoring procedure provides a catalog-wide priority leaderboard suitable for operational use.

References

- [1] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [2] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [3] Wolpert, D. H. (1992). Stacked Generalization. *Neural Networks*, 5(2), 241–259.
- [4] Kaufman, S., Rosset, S., & Perlich, C. (2012). Leakage in Data Mining: Formulation, Detection, and Avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), 1–21.
- [5] He, H., & Garcia, E. A. (2009). Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284.